

Current Investigation State

This document preserves the current working investigation state for the active Dune:
Awakening self-hosted troubleshooting effort.

Evidence Boundary

Use only evidence from the uploaded 'logs.zip', files uploaded after that point, and user-provided prompts/messages after that upload. Do not import facts, assumptions, client details, player details, or conclusions from unrelated labels or older projects.

Known Environment From User-Provided Evidence

Hosting stack: Proxmox bare-metal host, Ubuntu 24.04 VM, AMP control panel, Docker behind AMP

CPU allocation: 16 CPU cores from AMD Ryzen 9 9950X3D

Memory allocation: 96 GB assigned to VM from 192 GB host memory

Network: data center hosting, 5 Gbps fiber, 5 static IPs

Example AMP instance path: /home/amp/.ampdata/instances/DuneAwakening01/

Prior Working Findings

The process-visible OS shown in game logs may reflect container or guest runtime context and should not be treated as proof of the physical host OS.

The environment owner confirmed the broader host stack separately: Proxmox bare metal, Ubuntu VM, AMP, Docker behind AMP.

A prior UID/GID mismatch caused UserEngine.ini write failures when host path ownership did not match the container's game user context.

RabbitMQ logs previously showed per-player queue delete failures where the queue was still in use.

Prior director logs showed many ClassicalInstancing groups with empty server lists, while known primary partitions such as Survival_1 and Overmap reported server IDs.

Latest Uploaded Evidence: 2026-05-25 Travel Attempt

A newly provided log excerpt shows a failed travel attempt from Overmap/Overland to 'Story_ArtOfKanly' / partition 13.

Sensitive values from the uploaded log, including server password and player/account identifiers, are intentionally not reproduced here.

Observed Flow

06:49:00 - Player is reported on Overland / Overmap context.

06:49:00 - Travel initiated to Story_ArtOfKanly_DuelingRing_Lobby.

06:49:00 - Travel Initialize stage reports success.

06:49:01 - Travel Request stage reports success.

06:49:01 - TravelQueue Story_ArtOfKanly receives the request with DestinationPartitionId=13 and OriginId=Overmap2.

06:49:01 - Director receives one-player travel request for Story_ArtOfKanly with instancingMode=ClassicalInstancing.

06:49:01 - Director creates a travel request and assigns queue token 5.

06:49:01 - Director returns TravelResponse Code=Queued, ServerState=Ready, ServerFull=True, DestinationPartitionId=null, ServerLoginToken empty, Queue length 1.

06:49:02 - Story_ArtOfKanly / partition 13 reports ready=true with no players.

06:49:04 - Client/source-side travel fails with Bgd Travel Failed! Error:5.

06:49:04 - Same request ID appears again in TravelQueue Story_ArtOfKanly, but OriginId changes to Nowhere.

What This Proves

The source-side travel flow begins successfully.
The request reaches the Story_ArtOfKanly travel queue.
The director recognizes the request as ClassicalInstancing.
The director creates a queue token for the request.
The destination partition 13 reports ready=true shortly after the request.
The director does not return a usable destination handoff to the player.

What This Does Not Yet Prove

It does not prove the Story_ArtOfKanly game process received a player connection.
It does not prove a UDP/network failure because no destination listener or packet evidence was included in the latest upload.
It does not prove a database failure because no database/persistence error appears in the uploaded window.
It does not prove RabbitMQ is losing the request because the request is visible in the travel queue.
It does not prove the destination map binary is crashing because partition 13 reports server state ready=true.

Current Interpretation

The latest evidence strengthens the theory that the failure is occurring inside the director/instancing travel allocation path before a usable destination login handoff is issued to the player.

The most important anomaly is that the destination partition reports ready, but the travel response still returns 'ServerFull=True', 'DestinationPartitionId=null', and an empty 'ServerLoginToken'. That means the source/client appears not to receive a concrete usable destination even though the destination's server state exists.

The second important anomaly is the repeated/same request ID appearing with 'OriginId=Nowhere' after the source-side travel failure. This may indicate orphaned, retried, or cleanup-path queue handling after the original Overmap-originated request fails.

Current Leading Hypotheses

1. ClassicalInstancing allocation/capacity state mismatch:
The director sees the destination server state as ready but still treats the destination as full or unavailable for handoff.
2. Travel queue handoff failure:
The request reaches the correct queue and receives a queue token, but the director does not issue a destination partition and login token before the source travel request fails.
3. Stale or inconsistent instancing metadata:
Prior evidence showed ClassicalInstancing groups with empty server lists. The latest evidence shows partition 13 reporting ready, but not being allocated as a usable destination.
4. Post-failure orphan/retry path:
The same request ID reappearing with OriginId=Nowhere after Error:5 suggests cleanup or

retry logic may be re-queuing or re-processing the failed request without its original source context.

5. Capacity flag/path defect for specific ClassicalInstancing maps:
ServerFull=True with no players in the destination state suggests that capacity may be calculated from stale queue/group state rather than active player count alone.

Current Confidence

Network/firewall root cause: low confidence from latest upload alone.
Destination map crash root cause: low confidence from latest upload alone.
RabbitMQ total message loss: low confidence because request is visible.
Director/instancing allocation or capacity mismatch: moderate-to-high confidence.
Destination login/prelogin never reached: likely, but not yet proven without destination logs.

Evidence Needed Next

1. Destination map logs for the Story_ArtOfKanly container/service covering 06:48:45-06:49:15 UTC.
2. Director logs filtered by request ID around 06:48:45-06:49:15 UTC.
3. RabbitMQ queue/consumer state captured during a controlled retry.
4. Active UDP listener output before/during/after the retry.
5. Current director/partition metadata showing map, partition_id, server_id, dimension_index, blocked, and label.
6. Any director DB or config values related to ClassicalInstancing capacity, hard cap, queue size, and partition assignment for Story_ArtOfKanly.
7. A comparison travel attempt to one known-working destination and one known-failing ClassicalInstancing destination.

Next Recommended Controlled Test

Run one controlled travel attempt to 'Story_ArtOfKanly' while capturing:

Director logs
Overmap/source logs
Story_ArtOfKanly destination logs
RabbitMQ queue/consumer snapshot
UDP listener output
Packet capture if possible

The target is to determine whether the destination ever receives a player login/prelogin/authorization attempt. If it does not, the failure is likely before destination connection and remains in director allocation, queue response, token issuance, or source handoff.

Focused Commands For Next Capture

Use actual service/container names discovered in the environment.

Director request-id trace

Run on: Docker host shell, only if the director is a container

```
REQ='7AC249A990DE4A33B621FCA65F780813'  
docker logs <DIRECTOR_SERVICE> --since '2026-05-25T06:48:45Z' 2>&1 | grep -F "$REQ" -C 30
```

Destination map trace

Run on: Docker host shell, only if the destination is a container

```
docker logs <STORY_ARTOFKANLY_CONTAINER> --since '2026-05-25T06:48:45Z' 2>&1 | \
grep -Ei
'PreLogin|VerifyFlsIdentity|VerifyFlsAuthorization|Travel|Login|Join|FinishSpawn|Disconnect|Error|Warning|7AC249A990DE4A33B621FCA
```

RabbitMQ queue state during retry

Run on: Docker host shell, only if RabbitMQ is a container

```
docker exec <RABBITMQ_SERVICE> rabbitmqctl list_queues -p /\
name messages messages_ready messages_unacknowledged consumers state | \
grep -Ei
'Story_ArtOfKanly|completionAgency_Story_ArtOfKanly|serverStateSink_Story_ArtOfKanly|travelAgency'
```

UDP listeners during retry

Run on: Linux host or Linux VM shell

```
while true; do
echo "==== $(date -u --iso-8601=seconds) ====="
sudo ss -uapn | grep -Ei 'Dune|777|778|779|780|781|788|789|790|791|792' || true
sleep 2
done | tee story-artofkanly-listeners-during.log
```

Short Developer-Facing Summary

A travel request from Overmap2 to Story_ArtOfKanly/partition 13 reaches the ClassicalInstancing director path and receives queue token 5. The director reports ServerState=Ready but also ServerFull=True, DestinationPartitionId=null, and ServerLoginToken empty. The destination partition reports ready=true with no players one second later. The source-side request then fails with Bgd Travel Failed! Error:5, and the same request ID reappears in the Story_ArtOfKanly travel queue with OriginId=Nowhere. Current evidence points to director/instancing allocation, stale capacity metadata, or handoff-token issuance before destination PreLogin.